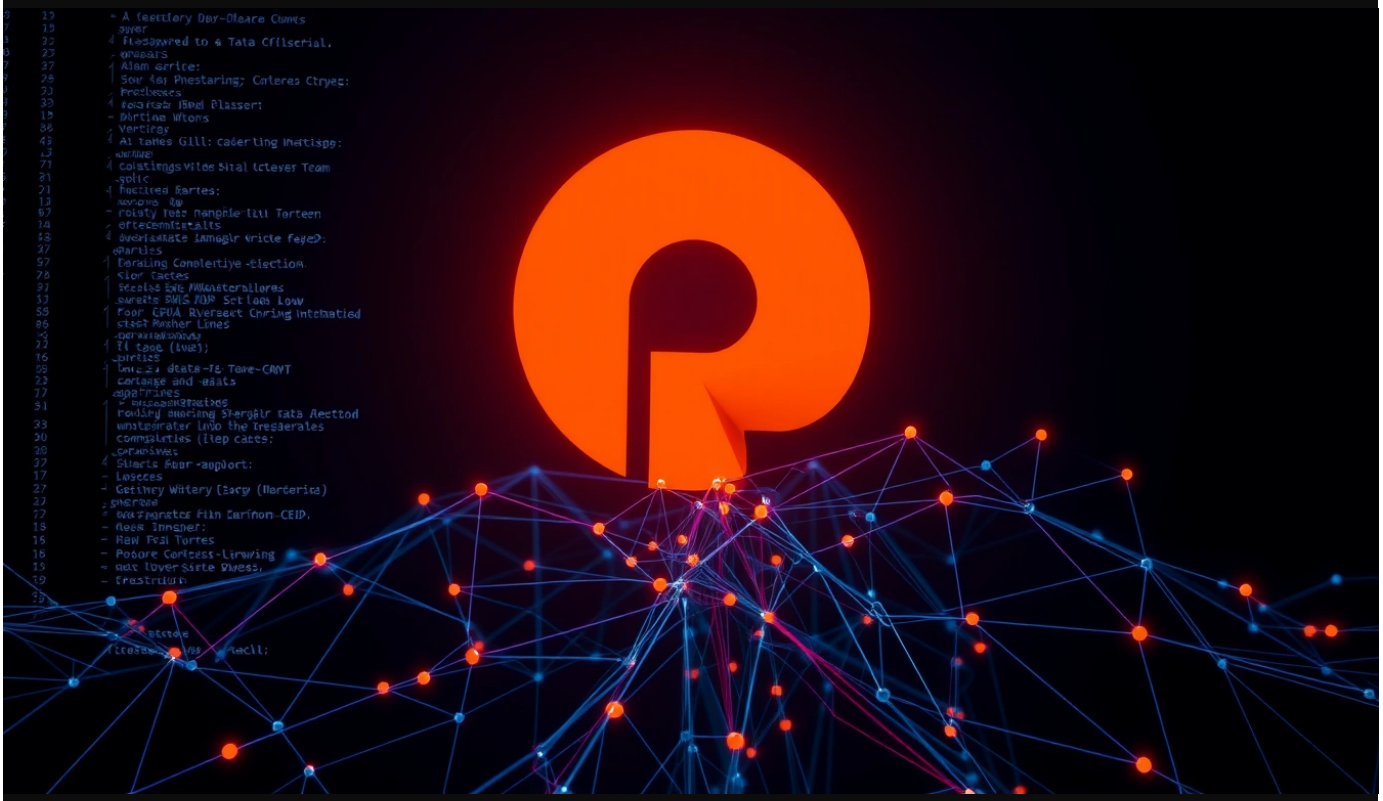

How to Run LLMs Locally with Ollama in 11 Steps [2026]

SOFIA LINDSTRÖM / APRIL 6, 2026 / AI & MACHINE LEARNING

Sofia Lindström April 6, 2026 25 min read

Running large language models locally gives you complete control over your data,

eliminates API costs, and lets you experiment with AI models offline. Ollama has emerged as the fastest way to get open-source LLMs running on your own hardware, with over 110,000 monthly searches from developers looking to run AI locally. This tutorial walks you through every step, from installation to building a Python-powered chatbot with a local LLM backend.

Table of Contents

Last updated: May 4, 2026

What's new as of May 2026: this walkthrough has been re-tested on the current Ollama stable release on macOS, Linux, and Windows. The install commands, REST API endpoints, Python library calls, and Docker Compose configuration below all reflect the runtime as it behaves today. If you followed an older version of this guide, the most likely changes that affect you are around concurrent request handling and the OpenAI-compatible `/v1/chat/completions` endpoint — both covered in detail later in this article.

By the end of this guide, you will have a fully functional local AI setup capable of running Llama 3.1, Mistral, Gemma 3, Qwen3, and dozens of other models on your own machine. You will also learn how to use the Ollama REST API, integrate with Python, deploy with Docker, and optimize performance for your specific hardware. Whether you are building privacy-sensitive applications, prototyping without API rate limits, or simply want to understand how LLMs work under the hood, this tutorial has you covered.

What Is Ollama and Why Run LLMs Locally?

Ollama is an open-source tool that lets you download, run, and manage large language models on your local machine. Think of it as Docker for AI models: you pull a model with a single command, and it handles quantization, memory management, and GPU acceleration automatically. Released in 2023 and now at version 0.13.x as of late 2025, Ollama has accumulated over 112 million model pulls for Llama 3.1 alone, making it the most popular local LLM runtime in the developer community.

The case for running LLMs locally is stronger than ever in 2026. OpenAI's GPT-5.4 API costs $15 per million input tokens. Anthropic's Claude Opus 4.6 charges $15 per million input tokens. For developers iterating on prompts, building chatbots, or processing sensitive documents, these costs add up fast. A local Llama 3.1 8B model running on Ollama costs exactly $0 per token, runs entirely offline, and keeps every byte of data on your hardware.

Privacy is the other major driver. Healthcare companies, law firms, and government contractors cannot send patient records, legal briefs, or classified information to third-party APIs. Running Ollama locally means your data never leaves your network. There are no terms of service to worry about, no data retention policies to audit, and no compliance risks from third-party processing. For enterprises subject to HIPAA, GDPR, or SOC 2 requirements, local LLMs are often the only viable option for AI-powered workflows.

Performance has improved dramatically as well. Open-source models like Llama 3.1 70B and Qwen3 32B now match or exceed GPT-4 on many benchmarks. Ollama can deliver 300+ tokens per second on consumer hardware with GPU acceleration, and up to 1,200 tokens per second on high-end setups. With quantized models, you can run a capable 8B parameter model on a laptop with just 8 GB of RAM.

Prerequisites and System Requirements

Before you begin, verify that your system meets the minimum requirements. Ollama runs on macOS, Linux, and Windows, but hardware requirements vary depending on the model size you plan to use. Below is a breakdown of what you need for each model tier.

Requirement	Minimum (7B models)	Recommended (13B-34B)	Advanced (70B+)
RAM	8 GB	32 GB	64 GB+
Disk Space	12 GB free	50 GB free	100 GB+ free
GPU (optional)	4 GB VRAM	12 GB VRAM	24 GB+ VRAM
OS	macOS 12+, Ubuntu 20.04+, Win 10+	Same	Same
CPU	4 cores, AVX2 support	8+ cores	16+ cores

Software prerequisites:

- **Ollama 0.13.x** (latest stable as of early 2026)
- **Python 3.10+** (for the Python integration steps)
- **Docker 24+** (optional, for containerized deployment)
- **NVIDIA drivers 535+** with CUDA 12.x (for NVIDIA GPU acceleration)

- **curl** or **wget** (for installation)
- A terminal emulator (Terminal.app, Windows Terminal, or any Linux terminal)

For GPU acceleration, NVIDIA cards from the GTX 1060 onwards are supported. AMD GPU support is available on Linux via ROCm 6.x. Apple Silicon Macs (M1 through M5) get automatic Metal GPU acceleration with no additional configuration. If you do not have a dedicated GPU, Ollama falls back to CPU inference, which is slower but still functional for smaller models.

Step 1: Install Ollama on Your System

Ollama provides one-line installers for every major platform. The installation process takes under two minutes and requires no compilation or dependency management.

On macOS and Linux, open your terminal and run:

```
 # Install Ollama (macOS and Linux)
curl -fsSL https://ollama.com/install.sh | sh

# Verify installation
ollama --version
# Expected output: ollama version 0.13.x
```

On macOS, the installer also adds a menu bar icon and a native desktop application that launched in July 2025. This app supports drag-and-drop for PDFs and images, a context-length slider, and a built-in model manager.

On Windows, download the installer from the Ollama website or use winget:

```
# Install Ollama on Windows via winget
winget install Ollama.Ollama

# Or download from https://ollama.com/download/windows
# Run the .exe installer and follow the prompts

# Verify installation (open a new terminal)
ollama --version
```

After installation, the Ollama service starts automatically in the background. It listens on `localhost:11434` by default. You can verify the service is running by hitting the health endpoint:

```
# Check if Ollama server is running
curl http://localhost:11434
# Expected output: Ollama is running
```

Common pitfall #1: On Linux, if you get a "permission denied" error, the install script may need sudo. Run `curl -fsSL https://ollama.com/install.sh | sudo sh` instead. On systems using systemd, Ollama installs as a service. Check its status with `systemctl status ollama`.

Step 2: Pull and Run Your First Model

With Ollama installed, you can download and run a model with a single command. The `ollama run` command pulls the model if it is not already cached locally, then starts an interactive chat session.

```
 # Pull and run Llama 3.1 8B (4.7 GB download)
ollama run llama3.1

# You'll see a download progress bar, then a prompt:
# >>> Send a message (/? for help)

# Try a prompt:
# >>> Explain quantum computing in 3 sentences
# Quantum computing uses qubits that can exist in multiple states
# simultaneously through superposition, enabling parallel processing
# of complex calculations. Unlike classical bits that are either 0 or 1,
# qubits use quantum entanglement to solve problems exponentially
# faster for specific tasks like cryptography and molecular simulation.
# Current quantum computers from IBM and Google have reached over 1,000
# qubits, though practical quantum advantage remains limited to
# specialized workloads.

# Exit the chat
# >>> /bye
```

The first run downloads the model weights, which can take several minutes depending on your internet speed. Subsequent runs start instantly because the model is cached in `~/.ollama/models`. The 8B parameter version of Llama 3.1 requires approximately 4.7 GB of disk space in its default Q4_0 quantization.

Common pitfall #2: If the download stalls or fails, it is usually a network timeout. Ollama does not resume partial downloads by default. Delete the incomplete model with `ollama rm llama3.1` and try again. On slow connections, consider pulling a smaller model first, such as `ollama run phi3` (2.3 GB).

Step 3: Explore the Model Library

Ollama hosts a curated library of over 100 open-source models, each optimized for local inference. Choosing the right model depends on your use case, available hardware, and performance requirements. Here is a comparison of the most popular models available as of early 2026.

Model	Parameters	Size (Q4)	Best For	Total Pulls
llama3.1	8B / 70B / 405B	4.7 GB / 40 GB / 230 GB	General purpose, coding	112.7M
mistral	7B	4.1 GB	Instruction following	35M+
gemma3	2B / 9B / 27B	1.6 GB / 5.4 GB / 16 GB	Multilingual, compact	18M+
qwen3	0.5B – 32B	0.4 GB – 19 GB	Reasoning, math, code	13.7M
codellama	7B / 13B / 34B	3.8 GB / 7.4 GB / 19 GB	Code generation	22M+
phi3	3.8B / 14B	2.3 GB / 7.9 GB	Efficient, edge devices	8M+
mistral-small3.2	24B	14 GB	Vision, tool calling	1.6M

deepseek-r1	7B / 14B / 70B	4.1 GB / 8.2 GB / 40 GB	Advanced reasoning	25M+

To list all locally available models and manage your collection:

```
 # List downloaded models
ollama list
# NAME            ID           SIZE    MODIFIED
# llama3.1:latest 62757c860e01 4.7 GB  2 hours ago
# mistral:latest  2ae6f6dd7a3d 4.1 GB  1 day ago

# Pull a specific model version
ollama pull gemma3:9b

# Pull a specific quantization
ollama pull llama3.1:70b-instruct-q4_K_M

# Show detailed model information
ollama show llama3.1 --verbose

# Delete a model to free space
ollama rm codellama
```

For most developers starting out, `llama3.1` (8B) offers the best balance of quality and speed. If you need a smaller model for a resource-constrained environment, `phi3` at 3.8B parameters delivers surprisingly good results in just 2.3 GB. For coding tasks specifically, `codellama` or `qwen3` outperform general-purpose models. If you need vision capabilities (image understanding), `mistral-small3.2` or `gemma3` support multimodal inputs.

Common pitfall #3: Pulling the 70B or 405B parameter versions without enough RAM will cause Ollama to crash or produce extremely slow output. Check your

available memory before pulling large models. On macOS, run `sysctl hw.memsize`; on Linux, run `free -h`. A 70B model needs at least 40 GB of RAM for acceptable performance.

Step 4: Use the Ollama REST API

Ollama exposes a REST API on `localhost:11434` that is compatible with the OpenAI API format, making it easy to integrate with existing tools and applications. This API supports chat completions, text generation, embeddings, and model management – all without authentication.

```
 # Generate a chat completion
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.1",
  "messages": [
    {"role": "system", "content": "You are a helpful coding assistant."},
    {"role": "user", "content": "Write a Python function to check if a num
  ],
  "stream": false
}'

# Response (abbreviated):
# {
#   "model": "llama3.1",
#   "message": {
#     "role": "assistant",
#     "content": "def is_prime(n):\n    if n < 2:\n        return False\n
#   },
#   "total_duration": 1245000000,
#   "eval_count": 142
# }

# Generate embeddings for RAG applications
```

```
curl http://localhost:11434/api/embed -d '{
  "model": "llama3.1",
  "input": "Ollama makes running LLMs locally simple and fast."
}'

# List available models via API
curl http://localhost:11434/api/tags
```

The API response includes performance metadata like `total_duration` (in nanoseconds), `eval_count` (tokens generated), and `eval_duration`. You can calculate tokens per second by dividing `eval_count` by `eval_duration`. On a machine with an NVIDIA RTX 4090, expect around 80-120 tokens per second for Llama 3.1 8B; on an Apple M3 Pro, around 40-60 tokens per second.

The streaming mode (`"stream": true`, which is the default) sends tokens as they are generated, just like ChatGPT's typing effect. Each token arrives as a separate JSON line, making it ideal for real-time chat interfaces. Set `"stream": false` when you need the complete response in a single JSON object, such as in batch processing pipelines.

Common pitfall #4: The Ollama API binds to `localhost` only by default. If you try to access it from another machine on your network, the connection will be refused. To expose the API on all interfaces, set the environment variable `OLLAMA_HOST=0.0.0.0` before starting the server. Be aware this opens the API to your entire network without authentication.

Step 5: Integrate Ollama with Python

The official Ollama Python library provides a clean, type-safe interface for

interacting with local models. This is where Ollama truly shines for developers building applications. Install it with pip and start generating responses in under five lines of code.

```
 # Install the official Ollama Python library
# pip install ollama

import ollama

# Simple chat completion
response = ollama.chat(
    model='llama3.1',
    messages=[
        {'role': 'system', 'content': 'You are a senior Python developer.'
        {'role': 'user', 'content': 'Explain decorators with a practical e
    ]
)
print(response['message']['content'])

# Streaming responses for real-time output
stream = ollama.chat(
    model='llama3.1',
    messages=[{'role': 'user', 'content': 'Write a haiku about debugging.'
    stream=True,
)
for chunk in stream:
    print(chunk['message']['content'], end='', flush=True)

# Generate embeddings for semantic search
embedding = ollama.embed(
    model='llama3.1',
    input='How do I configure Nginx as a reverse proxy?'
)
print(f"Embedding dimension: {len(embedding['embeddings'][0])}")

# List all local models programmatically
```

```
models = ollama.list()
for model in models['models']:
    print(f"{model['name']}: {model['size'] / 1e9:.1f} GB")
```

The Python library mirrors the REST API exactly but adds type hints, automatic serialization, and connection management. It handles reconnection if the Ollama server restarts and raises clear exceptions when models are not found or when the server is unreachable. For production applications, wrap calls in try-except blocks to handle `ollama.ResponseError` gracefully.

You can also use the OpenAI Python SDK with Ollama by pointing it at the local endpoint. This is useful if you have existing code that uses the OpenAI API and want to switch to a local model without rewriting your application:

```
# Using Ollama with the OpenAI Python SDK
# pip install openai

from openai import OpenAI

client = OpenAI(
    base_url='http://localhost:11434/v1',
    api_key='ollama',  # Required but not validated
)

response = client.chat.completions.create(
    model='llama3.1',
    messages=[
        {'role': 'user', 'content': 'What is the capital of France?'}
    ],
    temperature=0.7,
    max_tokens=200,
)
print(response.choices[0].message.content)
```

Common pitfall #5: If you get `ConnectionError: Connection refused` in Python, the Ollama server is not running. Start it with `ollama serve` in a separate terminal, or ensure the systemd service is active on Linux (`sudo systemctl start ollama`). On macOS, launch the Ollama desktop app.

Step 6: Build a Local Chatbot Application

Now let us build a complete, working chatbot that runs entirely on your machine. This project uses Ollama as the backend and a simple Python command-line interface with conversation history, system prompts, and streaming output. You can extend this into a web application later.

```
 #!/usr/bin/env python3
"""Local AI Chatbot powered by Ollama — Complete Working Example"""

import ollama
import sys
import json
from pathlib import Path

HISTORY_FILE = Path.home() / ".local_chatbot_history.json"
DEFAULT_MODEL = "llama3.1"
SYSTEM_PROMPT = """You are a helpful, knowledgeable AI assistant running
locally via Ollama. You provide concise, accurate answers. When asked abou
code, include working examples. When unsure, say so honestly."""

def load_history():
    """Load conversation history from disk."""
    if HISTORY_FILE.exists():
        with open(HISTORY_FILE) as f:
            return json.load(f)
```

```
    return []

def save_history(messages):
    """Persist conversation history to disk."""
    # Keep last 50 messages to avoid context overflow
    trimmed = messages[-50:]
    with open(HISTORY_FILE, 'w') as f:
        json.dump(trimmed, f, indent=2)

def get_available_models():
    """List all locally available models."""
    try:
        models = ollama.list()
        return [m['name'] for m in models['models']]
    except Exception:
        return []

def chat(model: str, messages: list, user_input: str):
    """Send a message and stream the response."""
    messages.append({'role': 'user', 'content': user_input})

    full_response = ""
    stream = ollama.chat(
        model=model,
        messages=messages,
        stream=True,
    )

    for chunk in stream:
        token = chunk['message']['content']
        print(token, end='', flush=True)
        full_response += token

    print()  # Newline after response
    messages.append({'role': 'assistant', 'content': full_response})
    return messages

def main():
```

```
    model = sys.argv[1] if len(sys.argv) > 1 else DEFAULT_MODEL

    # Verify model is available
    available = get_available_models()
    if available and model not in [m.split(':')[0] for m in available]:
        print(f"Model '{model}' not found. Available: {', '.join(available
        print(f"Pull it with: ollama pull {model}")
        sys.exit(1)

    print(f"Local AI Chatbot | Model: {model} | Type 'quit' to exit")
    print(f"Commands: /clear (reset), /model  (switch), /history (show)")
    print("-" * 60)

    messages = load_history()
    if not messages:
        messages = [{'role': 'system', 'content': SYSTEM_PROMPT}]

    while True:
        try:
            user_input = input("\nYou: ").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nGoodbye!")
            break

        if not user_input:
            continue
        if user_input.lower() == 'quit':
            save_history(messages)
            print("Chat saved. Goodbye!")
            break
        if user_input == '/clear':
            messages = [{'role': 'system', 'content': SYSTEM_PROMPT}]
            print("Conversation cleared.")
            continue
        if user_input == '/history':
            print(f"Messages in context: {len(messages)}")
            continue
        if user_input.startswith('/model '):
```

```
            model = user_input.split(' ', 1)[1]
            print(f"Switched to model: {model}")
            continue

        print(f"\n{model}: ", end='')
        messages = chat(model, messages, user_input)
        save_history(messages)

if __name__ == '__main__':
    main()
```

Save this as `chatbot.py` and run it with `python3 chatbot.py`. You can optionally specify a different model: `python3 chatbot.py mistral`. The chatbot persists conversation history to disk, so you can close the terminal and resume later. It streams tokens in real time, giving the same responsive feel as cloud-based chat interfaces.

This is a fully working project you can extend. Add a Flask or FastAPI frontend to turn it into a web app. Integrate it with a RAG pipeline using LangChain to query your own documents. Or connect it to a Slack bot for team-wide access to a private AI assistant.

Step 7: Create Custom Models with Modelfiles

Ollama's Modelfile system lets you create customized model variants with specific system prompts, parameters, and behaviors. This is similar to Dockerfiles: you define a base model, set configuration, and build a new named model that you can run repeatedly.

```
# Save as Modelfile.code-reviewer
```

```
FROM llama3.1

# Set the system prompt
SYSTEM """You are an expert code reviewer. When given code, you:
1. Identify bugs and security vulnerabilities
2. Suggest performance improvements
3. Check for best practices and clean code principles
4. Rate the code quality from 1-10
Always be constructive and explain WHY something should change."""

# Adjust generation parameters
PARAMETER temperature 0.3
PARAMETER top_p 0.9
PARAMETER num_ctx 8192
PARAMETER stop "<|eot_id|>"
```

Build and run your custom model:

```
 # Create the custom model
ollama create code-reviewer -f Modelfile.code-reviewer
# transferring model data
# creating model layer
# writing manifest
# success

# Run the custom model
ollama run code-reviewer
# >>> Review this Python function:
# >>> def get_user(id):
# >>>     conn = sqlite3.connect("db.sqlite")
# >>>     return conn.execute(f"SELECT * FROM users WHERE id = {id}").fetc

# The model will identify the SQL injection vulnerability,
# missing connection cleanup, and other issues.
```

The `PARAMETER num_ctx` setting controls the context window size. The default is 2048 tokens, which is often too small for code review or document analysis. Increasing it to 8192 or 16384 lets the model process longer inputs, but requires proportionally more memory. A 8192-token context on an 8B model adds roughly 1 GB of additional RAM usage.

You can create multiple Modelfiles for different tasks: a SQL query generator, a technical writer, a commit message author, or a test case generator. Each custom model is stored as a lightweight layer on top of the base model, so creating ten custom variants of Llama 3.1 does not use ten times the disk space.

Step 8: Deploy Ollama with Docker

Docker deployment is essential for team environments, CI/CD pipelines, and production servers. Ollama provides official Docker images with GPU support, making it straightforward to containerize your local AI setup.

```
 # docker-compose.yml - Ollama with GPU support
version: '3.8'

services:
  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    deploy:
      resources:
        reservations:
```

```
            devices:
              - driver: nvidia
                count: all
                capabilities: [gpu]
      restart: unless-stopped
      environment:
        - OLLAMA_HOST=0.0.0.0

    # Optional: Web UI for Ollama
    open-webui:
      image: ghcr.io/open-webui/open-webui:main
      container_name: open-webui
      ports:
        - "3000:8080"
      volumes:
        - open_webui_data:/app/backend/data
      environment:
        - OLLAMA_BASE_URL=http://ollama:11434
      depends_on:
        - ollama
      restart: unless-stopped

  volumes:
    ollama_data:
    open_webui_data:
```

Launch the stack with `docker compose up -d`, then pull models inside the container:

```
   # Start the Docker stack
  docker compose up -d

  # Pull a model inside the container
  docker exec -it ollama ollama pull llama3.1
```

```
# Verify the API is accessible
curl http://localhost:11434/api/tags

# Access the web UI at http://localhost:3000
```

The Docker Compose file above includes Open WebUI, a full-featured chat interface that looks and feels like ChatGPT but runs entirely locally. It supports multiple models, conversation history, file uploads, and user management. This setup gives your entire team access to local LLMs through a web browser without installing anything on their machines.

For CPU-only Docker deployments (no NVIDIA GPU), remove the `deploy.resources` section from the compose file. If you are new to Docker, the CPU-only configuration works identically, just slower for inference.

Step 9: Optimize Performance and Memory Usage

Getting the best performance out of Ollama requires understanding how quantization, context length, and GPU offloading affect speed and memory. Here are the key optimizations you should apply based on your hardware.

Choose the right quantization level. Quantization reduces model precision to save memory and increase speed, with minimal quality loss. Ollama uses Q4_0 by default, which reduces a 16-bit model to 4-bit precision. For most use cases, Q4_K_M offers a better quality-speed tradeoff:

Quantization	Bits per Weight	Llama 3.1 8B Size	Quality Impact	Speed (tokens/s)

F16 (full)	16	16 GB	Baseline	25-40
Q8_0	8	8.5 GB	Negligible	45-70
Q6_K	6	6.6 GB	Minimal	55-80
Q4_K_M	4	4.9 GB	Slight	70-110
Q4_0	4	4.7 GB	Slight	75-120
Q2_K	2	3.2 GB	Noticeable	90-140

GPU layer offloading. Ollama automatically detects your GPU and offloads as many model layers as VRAM allows. You can control this manually for fine-grained optimization:

```
 # Set the number of GPU layers (higher = more VRAM used, faster)
OLLAMA_NUM_GPU=35 ollama run llama3.1

# Force CPU-only mode (useful for benchmarking)
OLLAMA_NUM_GPU=0 ollama run llama3.1

# Increase context window (default 2048)
ollama run llama3.1 --num-ctx 8192

# Monitor GPU usage during inference
# NVIDIA: nvidia-smi -l 1
# AMD: rocm-smi
# Apple: sudo powermetrics --samplers gpu_power
```

Memory management tips. Ollama keeps models loaded in memory for 5 minutes after the last request by default. For servers handling multiple models, you can

reduce this timeout to free resources faster:

```
 # Reduce model keep-alive time to 60 seconds
OLLAMA_KEEP_ALIVE=60s ollama serve

# Or set it per-request via the API
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.1",
  "messages": [{"role": "user", "content": "Hello"}],
  "keep_alive": "30s"
}'

# Unload a model immediately
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.1",
  "keep_alive": 0
}'
```

Concurrent request handling. Ollama supports parallel inference as of version 0.13.x. By default, it handles one request at a time and queues additional requests. To enable concurrent processing, set `OLLAMA_NUM_PARALLEL=4` (adjust based on available VRAM). Each parallel slot requires additional memory roughly equal to the context window size times the number of slots.

Step 10: Set Up Ollama as a Development Tool

Ollama integrates with popular developer tools to provide AI-powered assistance without sending code to external servers. Here are three high-value integrations you can set up in minutes.

VS Code with Continue extension. The Continue extension (continue.dev) connects

directly to Ollama, giving you tab completion, inline chat, and code explanation powered by your local model. Install the extension, then add this to your Continue config:

```
{
  "models": [
    {
      "title": "Ollama Llama 3.1",
      "provider": "ollama",
      "model": "llama3.1",
      "apiBase": "http://localhost:11434"
    }
  ],
  "tabAutocompleteModel": {
    "title": "Ollama CodeLlama",
    "provider": "ollama",
    "model": "codellama"
  }
}
```

Git commit message generation. Use Ollama to automatically generate commit messages from your staged changes. Add this as a shell function in your `.bashrc` or `.zshrc`:

```
# Add to ~/.bashrc or ~/.zshrc
ai-commit() {
  local diff=$(git diff --cached)
  if [ -z "$diff" ]; then
    echo "No staged changes. Run 'git add' first."
    return 1
  fi

  local msg=$(echo "$diff" | ollama run llama3.1 \
```

```
    "Generate a concise git commit message (max 72 chars first line) for t

  echo "Suggested: $msg"
  read -p "Use this message? (y/n/e[dit]) " choice
  case $choice in
    y) git commit -m "$msg" ;;
    e) git commit -e -m "$msg" ;;
    *) echo "Commit cancelled." ;;
  esac
}

# Usage: stage changes, then run ai-commit
```

Local API for shell scripts. Pipe terminal output through Ollama for instant analysis. This is especially useful for parsing error logs, explaining command output, or generating documentation:

```
 # Explain a complex error
kubectl logs my-pod --tail=50 2>&1 | ollama run llama3.1 \
  "Explain what's wrong based on these Kubernetes logs and suggest a fix:"

# Generate documentation from code
cat src/main.py | ollama run llama3.1 \
  "Generate API documentation in Markdown for this Python module:"

# Analyze test failures
pytest --tb=short 2>&1 | ollama run llama3.1 \
  "Summarize the test failures and suggest fixes:"
```

These integrations demonstrate why running an LLM locally with Ollama is transformative for developer workflows. Every query stays on your machine, there is zero latency from network round trips to cloud APIs, and you can pipe any data – including proprietary code – without security concerns. For teams using AI coding

tools like GitHub Copilot or Cursor, Ollama provides a self-hosted alternative that works offline.

Step 11: Run Multiple Models and A/B Test Responses

One of the biggest advantages of local LLMs is the ability to run multiple models side-by-side and compare outputs. This is invaluable for prompt engineering, model evaluation, and choosing the best model for a specific task.

```
 #!/usr/bin/env python3
"""Compare responses from multiple local models."""

import ollama
import time

MODELS = ['llama3.1', 'mistral', 'gemma3:9b']
PROMPT = "Explain the difference between TCP and UDP in exactly 3 sentence

for model_name in MODELS:
    print(f"\n{'='*60}")
    print(f"Model: {model_name}")
    print('='*60)

    start = time.time()
    response = ollama.chat(
        model=model_name,
        messages=[{'role': 'user', 'content': PROMPT}],
    )
    elapsed = time.time() - start

    content = response['message']['content']
    tokens = response.get('eval_count', 0)
```

```
        print(content)
        print(f"\n[Tokens: {tokens} | Time: {elapsed:.1f}s | Speed: {tokens/el
```

Running this script produces side-by-side comparisons with performance metrics. You can evaluate which model gives more accurate, concise, or detailed responses for your specific prompts. For systematic evaluation, extend the script to test against a dataset of prompts and score responses automatically using another model as a judge (the "LLM-as-judge" pattern). This approach is used by leading AI research labs and is directly applicable to local development with Ollama.

Troubleshooting: 8 Common Issues and Fixes

Even with Ollama's simple setup, you may encounter issues. Here are the eight most common problems and their solutions, based on GitHub issues and community reports.

1. "Error: model not found" after pulling. This usually means the model name is misspelled or the pull was interrupted. Run `ollama list` to see what is actually available. Model names are case-sensitive and require exact tags: `llama3.1`, not `Llama3.1` or `llama-3.1`.

2. Ollama server not starting on Linux. Check the systemd service: `sudo systemctl status ollama`. Common causes include port 11434 already in use (check with `lsof -i :11434`) or the ollama user not having permission to access the GPU. Fix GPU permissions with `sudo usermod -aG render,video $USER`.

3. Out of memory (OOM) crashes. Ollama 0.13.x includes improved memory estimation, but OOM errors still occur when running models too large for your

system. Solutions: use a smaller quantization (`ollama pull llama3.1:8b-instruct-q2_K`), reduce context length (`--num-ctx 1024`), or close memory-heavy applications before running large models.

4. Extremely slow generation (under 5 tokens/s). This indicates CPU-only inference when you expected GPU acceleration. Verify GPU detection: `ollama ps` shows the processor column. For NVIDIA, ensure CUDA toolkit and drivers are installed. For AMD on Linux, install ROCm. On macOS with Apple Silicon, Metal acceleration should be automatic – if it is not working, update to the latest macOS version.

5. Model downloads failing or corrupting. Large model downloads (40+ GB) can fail on unstable connections. Ollama does not support resume, so a failed download must restart from zero. Workarounds: use a wired connection, increase TCP keepalive settings, or pull during off-peak hours. If a model seems corrupted (garbled output), delete it with `ollama rm model_name` and re-pull.

6. API returning 404 on /v1/chat/completions. The OpenAI-compatible endpoint is at `/v1/chat/completions`, but Ollama's native endpoint is at `/api/chat`. If you are using the OpenAI SDK, set `base_url='http://localhost:11434/v1'`. If you are using curl directly with Ollama's native format, use `/api/chat` instead.

7. Docker container cannot access GPU. NVIDIA Container Toolkit must be installed for GPU passthrough to Docker. Install it with `sudo apt install nvidia-container-toolkit` and restart Docker. Verify with `docker run --rm --gpus all nvidia/cuda:12.0-base nvidia-smi`. If this fails, your Docker installation does not have GPU support configured.

8. Models producing repetitive or low-quality output. Adjust generation parameters. Increase `temperature` (0.7-1.0) for more creative output. Set

`repeat_penalty` to 1.2-1.5 to reduce repetition. Add a `top_k` of 40 and `top_p` of 0.9. These can be set in Modelfiles or passed via the API. If quality is consistently poor, try a larger model or a different model family.

Advanced Tips for Power Users

Once you have Ollama running reliably, these advanced techniques will help you get more out of your local AI setup.

Run Ollama behind Nginx for team access. Expose Ollama to your local network with TLS and basic authentication by placing it behind an Nginx reverse proxy. This lets multiple team members share a single GPU server for inference:

```
 # /etc/nginx/conf.d/ollama.conf
server {
    listen 443 ssl;
    server_name ai.internal.company.com;

    ssl_certificate /etc/ssl/certs/ollama.crt;
    ssl_certificate_key /etc/ssl/private/ollama.key;

    auth_basic "AI Server";
    auth_basic_user_file /etc/nginx/.htpasswd;

    location / {
        proxy_pass http://127.0.0.1:11434;
        proxy_set_header Host $host;
        proxy_read_timeout 300s;
        proxy_buffering off;  # Required for streaming
    }
}
```

Use environment variables for persistent configuration. Instead of passing flags on every command, set these in your shell profile or systemd service file:

```
# Add to ~/.bashrc or /etc/environment
export OLLAMA_HOST=0.0.0.0          # Listen on all interfaces
export OLLAMA_MODELS=/data/ollama   # Custom model storage path
export OLLAMA_NUM_PARALLEL=4        # Handle 4 concurrent requests
export OLLAMA_MAX_LOADED_MODELS=2   # Keep 2 models in memory
export OLLAMA_KEEP_ALIVE=5m         # Unload models after 5 min idle
export OLLAMA_NUM_GPU=99            # Use all available GPU layers
export OLLAMA_FLASH_ATTENTION=1     # Enable flash attention (faster)
```

Build a local RAG pipeline. Combine Ollama's embedding API with a vector database like ChromaDB to build a retrieval-augmented generation system that answers questions from your own documents. This is the same architecture used by enterprise AI products, running entirely on your hardware. For a detailed walkthrough, see our RAG chatbot tutorial with LangChain.

Monitor resource usage. On long-running servers, track Ollama's memory and GPU usage over time. Use `ollama ps` to see which models are loaded, their memory footprint, and the processor being used. Combine this with Prometheus and Grafana for production monitoring. The Ollama API exposes `/api/ps` endpoint that returns JSON-formatted resource usage data suitable for scraping.

Scheduled model updates. New model versions are released frequently. Set up a cron job to pull updated models weekly:

```
# Add to crontab: crontab -e
# Update models every Sunday at 3 AM
0 3 * * 0 /usr/local/bin/ollama pull llama3.1 >> /var/log/ollama-update.lo
```

```
0 3 * * 0 /usr/local/bin/ollama pull mistral >> /var/log/ollama-update.log
```

Security Considerations for Local LLMs

Running AI models locally shifts the security perimeter from a cloud provider to your own infrastructure. While this eliminates data exfiltration risks to third parties, it introduces new considerations you need to address.

Network exposure. By default, Ollama binds to localhost only, which is secure for single-user setups. If you change `OLLAMA_HOST` to `0.0.0.0` for network access, the API has no authentication. Anyone on your network can send prompts, list models, or delete models. Always place a reverse proxy with authentication in front of any network-exposed Ollama instance.

Model provenance. Only pull models from Ollama's official library or trusted sources. Third-party GGUF files could theoretically contain adversarial weights designed to produce harmful outputs for specific prompts. Stick to verified models from Meta (Llama), Google (Gemma), Mistral AI, and other established providers available through `ollama pull`.

Prompt injection. Local models are just as susceptible to prompt injection as cloud models. If your application passes user input to Ollama, sanitize and validate inputs. Use system prompts to constrain model behavior, and never execute code generated by the model without human review. This applies equally to agentic AI applications built on local models.

Data persistence. Ollama does not log prompts or responses by default, but the conversation history in custom applications (like the chatbot we built in Step 6)

persists on disk. Encrypt the storage volume if the data is sensitive, and implement appropriate access controls. For compliance-regulated environments, add audit logging to track who queried which model with what input.

Ollama vs Cloud AI APIs: When to Use Each

Running LLMs locally is not always the right choice. Understanding when to use Ollama versus cloud APIs helps you make the right architectural decision for each project.

Use Ollama when: data privacy is non-negotiable, you need offline capability, your use case involves high-volume inference that would be cost-prohibitive with APIs (processing thousands of documents), you are prototyping and iterating rapidly on prompts, or you want to fine-tune and customize models. Development teams that process more than 10 million tokens per month typically break even on hardware costs versus API pricing within 3-6 months.

Use cloud APIs when: you need the absolute best model quality (GPT-5.4, Claude Opus 4.6), your usage is sporadic and low-volume, you do not want to manage hardware, you need extremely long context windows (200K+ tokens), or you need real-time web search integration. For a detailed comparison of cloud AI options, see our Claude vs ChatGPT comparison.

Many production architectures use both: Ollama for high-volume, privacy-sensitive workloads and cloud APIs for complex tasks that require frontier model capabilities. The OpenAI-compatible API format means you can switch between local and cloud backends by changing a single URL in your configuration.

Related Coverage

- How to Build a RAG Chatbot with Python and LangChain
- How to Get Started with Docker: Complete Beginner Tutorial
- How to Configure Nginx as a Reverse Proxy
- GitHub Copilot vs Cursor 2026
- Claude vs ChatGPT 2026
- Agentic AI in Enterprise 2026
- How to Automate Tasks with Python

Frequently Asked Questions

Is Ollama free to use?

Yes, Ollama is completely free and open source under the MIT license. There are no usage limits, no API keys required for local use, and no telemetry or data collection. The models themselves are also free, released under licenses like Llama Community License (Meta), Apache 2.0 (Mistral, Gemma), and similar open-source licenses. Your only cost is the hardware you run it on.

Can I run Ollama without a GPU?

Yes. Ollama runs on CPU-only systems using optimized inference libraries (llama.cpp under the hood). Performance is slower – expect 5-15 tokens per second on a modern CPU versus 60-120 tokens per second on a mid-range GPU. For CPU-only setups, stick to smaller models (7B-8B parameters) with aggressive quantization (Q4_0 or Q2_K). A modern CPU with AVX2 support and 16 GB of RAM can run Llama 3.1 8B comfortably.

How much disk space do I need for Ollama?

Ollama itself is under 100 MB. Model sizes vary: a 7B model is typically 4-5 GB, a 13B model is 7-8 GB, a 34B model is 19-20 GB, and a 70B model is about 40 GB. If you plan to keep 3-4 models downloaded, allocate at least 30 GB of free disk space. Models are stored in `~/.ollama/models` by default, but you can change this with the `OLLAMA_MODELS` environment variable to use a different drive.

Can I fine-tune models with Ollama?

Ollama does not support fine-tuning directly. It is an inference runtime, not a training framework. For fine-tuning, use tools like Unsloth, Axolotl, or Hugging Face PEFT to train a LoRA adapter, export it as a GGUF file, and then import it into Ollama using a Modelfile with the `ADAPTER` instruction. This workflow lets you customize model behavior while still using Ollama for serving.

How does Ollama compare to LM Studio and GPT4All?

Ollama, LM Studio, and GPT4All all run local LLMs, but serve different audiences. Ollama is developer-focused with a CLI-first approach, a REST API, and Docker support – ideal for integration into applications and pipelines. LM Studio provides a polished GUI with a built-in chat interface, making it better for non-technical users. GPT4All emphasizes ease of use and a curated model selection. Ollama has the largest model library (100+ models), the most active community, and the best API compatibility with OpenAI's format.

Can multiple users share one Ollama server?

Yes. Set `OLLAMA_HOST=0.0.0.0` to listen on all interfaces, and users can connect via the API from any machine on the network. For concurrent requests, set `OLLAMA_NUM_PARALLEL` to the number of simultaneous users you want to support

(each parallel slot uses additional VRAM). For production team use, deploy Open WebUI alongside Ollama for user accounts, chat history, and a web-based interface.

Does Ollama support vision and multimodal models?

Yes. Models like `llava`, `mistral-small3.2`, and `gemma3` support image inputs. You can pass images via the API as base64-encoded data or file paths. In the CLI, use the `/image` command to attach an image to your prompt. The macOS desktop app supports drag-and-drop for images and PDFs directly into the chat window.

Is Ollama suitable for production workloads?

Ollama is suitable for production in environments where you control the hardware and traffic volume. It supports Docker deployment, health checks, concurrent requests, and automatic model loading. However, it lacks built-in features like request authentication, rate limiting, and horizontal scaling. For high-availability production deployments, place Ollama behind a load balancer with multiple GPU servers, and use Nginx or Traefik for authentication and rate limiting.

Sofia Lindström

EDITOR-IN-CHIEF

Sofia Lindström is the Editor-in-Chief at Tech Insider, where she leads editorial strategy and oversees coverage across AI, cybersecurity, and enterprise technology. With over a decade…

View all articles →

TECH INSIDER

Tech Insider delivers in-depth coverage of the technologies shaping the future: AI, cybersecurity, cloud computing, hardware, and the trends that matter.

editorial@tech-insider.org

COMPANY

About Us

Terms of Service

Privacy Policy

Contact

EXPLORE

AI & Machine Learning

Cloud Computing

Crypto Casino

Cybersecurity

Gaming

CATEGORIES

Hardware

iGaming Tech

Mystery Boxes

EN AU CA IT DE NL SV FR FI